



Tema 3

Recycler View



¿Que es?

El **RecyclerView** es una interfaz de usuario avanzada y flexible para mostrar grandes conjuntos de datos estructurados. Permite la creación eficiente de interfaces de usuario mediante el diseño de elementos en una lista y el **reciclaje** de vistas para ahorrar recursos.

El RecyclerView reemplaza a ListView y GridView y ofrece un mayor control sobre la disposición y animación de los elementos de la lista.

Pasos clave para implementar RecyclerView:

- Creación del view (RecyclerView)
- Diseño del Elemento de la Lista
- Creación del Adaptador
- Creación del ViewHolder

1º

Añadimos el
recyclerview a nuestro
layout

Importante añadir el
layout padre como
scrollView para que el
recyclerview pueda
scrollear y no se
quede estático

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <androidx.recyclerview.widget.RecyclerView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>

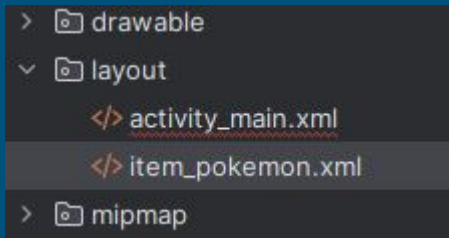
    </LinearLayout>

</ScrollView>
```



2º Ahora debemos crear una lista especializada para ese listado

Creamos el xml de cada "item":



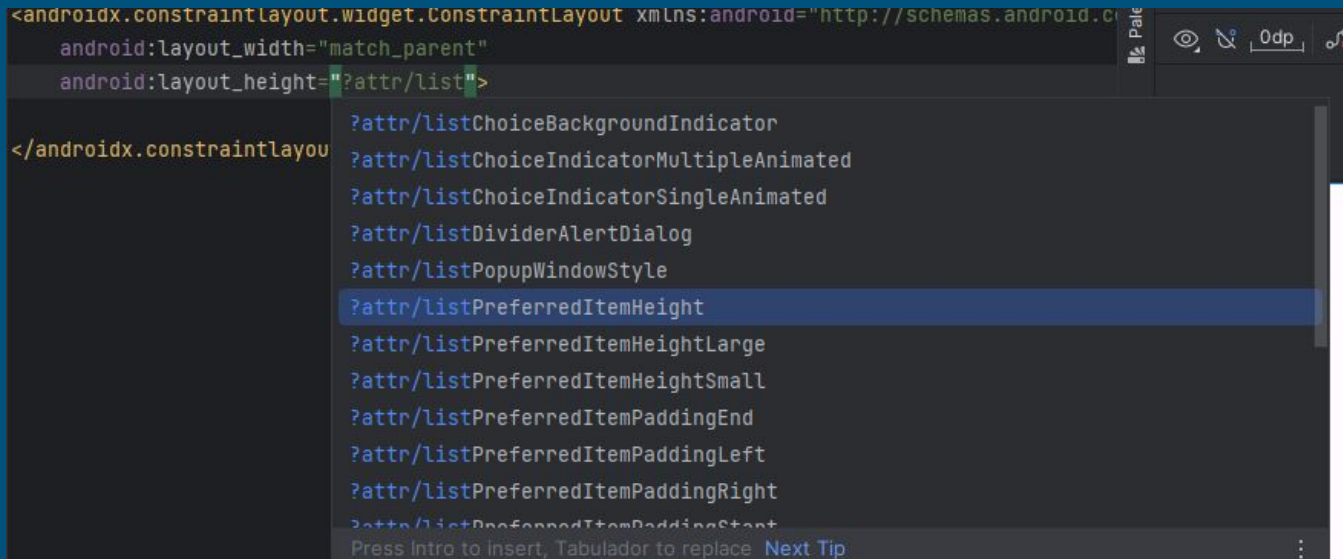
Debemos darle una altura a cada elemento sino cojeria todo el alto del padre.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="50dp">
</androidx.constraintlayout.widget.ConstraintLayout>
```



Podríamos darle altura de otra forma

Con estilos ya predefinidos:



```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="?attr/list">

</androidx.constraintlayout.widget.ConstraintLayout>
```

The dropdown menu lists the following attributes:

- ?attr/listChoiceBackgroundIndicator
- ?attr/listChoiceIndicatorMultipleAnimated
- ?attr/listChoiceIndicatorSingleAnimated
- ?attr/listDividerAlertDialog
- ?attr/listPopupWindowStyle
- ?attr/listPreferredItemHeight**
- ?attr/listPreferredItemHeightLarge
- ?attr/listPreferredItemHeightSmall
- ?attr/listPreferredItemPaddingEnd
- ?attr/listPreferredItemPaddingLeft
- ?attr/listPreferredItemPaddingRight
- ?attr/listPreferredItemPaddingStart

Press Intro to insert, Tabulador to replace Next Tip

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http:
3      android:layout_width="match_parent"
4      android:layout_height="?attr/listPreferredItemHeight"
5      xmlns:app="http://schemas.android.com/apk/res-auto"
6      android:background="@color/verde">
7
8      <CheckBox
9          android:layout_width="wrap_content"
10         android:layout_height="wrap_content"
11         android:id="@+id/checkBox"
12         app:layout_constraintStart_toStartOf="parent"
13         app:layout_constraintTop_toTopOf="parent"
14         app:layout_constraintBottom_toBottomOf="parent"
15         />
16
17     <TextView
18         android:id="@+id/textView"
19         android:layout_width="wrap_content"
20         android:layout_height="wrap_content"
21         android:layout_marginStart="8dp"
22         android:text="Pokemon"
23         android:textAppearance="?attr/textAppearanceListItem"
24         android:textColor="@color/white"
25         android:textSize="18sp"
26         app:layout_constraintBottom_toBottomOf="parent"
27         app:layout_constraintStart_toEndOf="@+id/checkBox"
28         app:layout_constraintTop_toTopOf="parent" />
29
30
31
32 </androidx.constraintlayout.widget.ConstraintLayout>

```



Pokemon

Ya tenemos creado como se van a ver
nuestro elementos

3º Actualizamos el recyclerView con la lista que vamos a usar

```
<androidx.recyclerview.widget.RecyclerView  
    android:id="@+id/recycler_view"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    tools:listitem="@layout/item_pokemon" />
```

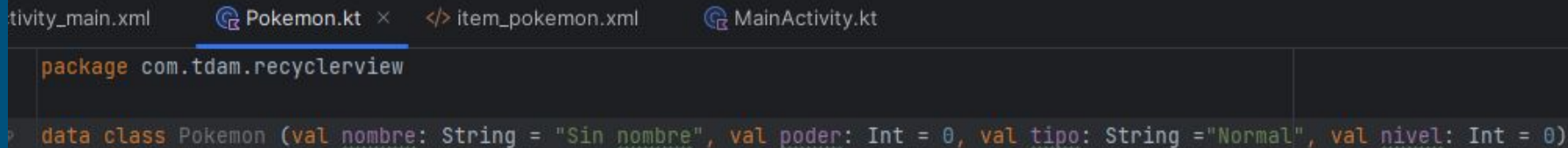


4º Crear dos archivos importantes

1- Una **clase** para almacenar el objeto que necesitamos para la lista de Pokemons.

En este caso Pokemon.kt.

Si la hacemos data class nos ahorramos trabajo



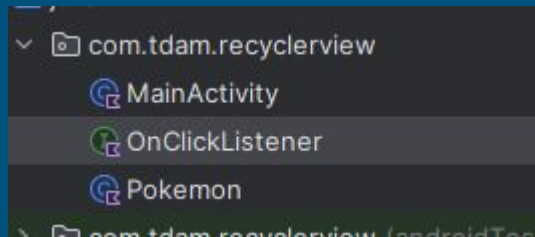
```
activity_main.xml  Pokemon.kt  x  </> item_pokemon.xml  MainActivity.kt

package com.tdam.recyclerview

data class Pokemon (val nombre: String = "Sin nombre", val poder: Int = 0, val tipo: String = "Normal", val nivel: Int = 0)
```

2 - Una **interfaz** que va a servir para poder comunicar la actividad con la clase encargada de manejar los listados de los recyclerView (adapters).

La llamaremos OnClickListener:



```
package com.tdam.recyclerview

interface OnClickListener {

    fun onLongClick(pokemon: Pokemon) //funcion que se ejecuta cuando se hace click largo en un elemento de la lista
}
```

La creación de una **interfaz** para un RecyclerView en Android se realiza comúnmente para gestionar las interacciones entre los elementos del **RecyclerView** y la **actividad** o fragmento que lo contiene.

La interfaz actúa como un contrato que define métodos que deben implementarse en la actividad o fragmento para manejar eventos específicos, como hacer clic en un elemento del RecyclerView.

Razones para usar una interfaz con RecyclerView

Desacoplamiento: Al utilizar una interfaz, se logra un desacoplamiento entre la lógica de la interfaz de usuario del RecyclerView y la lógica de la actividad o fragmento que lo contiene. Esto hace que el código sea más modular y fácil de mantener.

Reutilización: La interfaz permite reutilizar fácilmente el mismo RecyclerView con diferentes implementaciones de manejo de eventos en diversas partes de la aplicación.

Escalabilidad: A medida que la lógica de la aplicación crece, el uso de una interfaz facilita la adición de nuevas funcionalidades o cambios en la lógica de manejo de eventos sin afectar otras partes del código.

Legibilidad: La interfaz proporciona un contrato claro de los eventos que pueden ocurrir en el RecyclerView, lo que mejora la legibilidad del código y facilita la comprensión del flujo de datos y eventos.

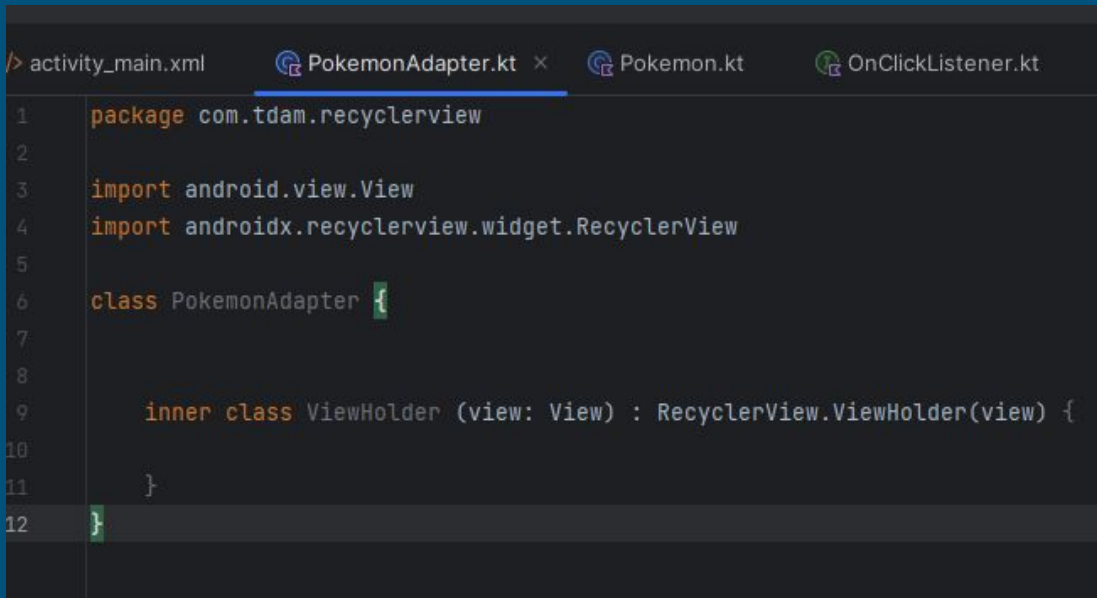
Creamos el adaptador para la lista

Esta clase nos va a permitir “inflar” la vista.

Basicamente coje el archivo `item_pokemon.xml` y lo crea dentro del `recyclerView` para cada elemento.

Es lo mismo que pasa con `setContentView` en el `onCreate`

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)   
    }  
}
```



```
> activity_main.xml  PokemonAdapter.kt  Pokemon.kt  OnClickListener.kt  
1  package com.tdam.recyclerview  
2  
3  import android.view.View  
4  import androidx.recyclerview.widget.RecyclerView  
5  
6  class PokemonAdapter {  
7  
8  
9      inner class ViewHolder (view: View) : RecyclerView.ViewHolder(view) {  
10  
11      }  
12 }
```

Antes de nada añadimos binding

```
build.gradle.kts (Module :app)  
proguard-rules.pro (ProGuard Rule)
```

```
buildFeatures { this: ApplicationBuildFeatures  
    viewBinding = true  
}
```

```
> class MainActivity : AppCompatActivity() {  
  
    private lateinit var binding: ActivityMainBinding  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
    }  
}
```

ViewHolder

Creamos una clase **ViewHolder** que extienda `RecyclerView.ViewHolder`.

Este será responsable de mantener referencias a las vistas individuales dentro de un elemento de la lista y evitar tener que realizar búsquedas repetitivas en la jerarquía de vistas cada vez que se recicla un elemento. Su función principal es almacenar referencias a los elementos de la interfaz de usuario dentro de cada elemento de la lista para que puedan ser reutilizados eficientemente.

Ejemplo:

```
class MiAdapter(private val datos: List<MiModelo>) : RecyclerView.Adapter<MiAdapter.ViewHolder>() {

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
        val view = LayoutInflater.from(parent.context).inflate(R.layout.item_mi_layout, parent, false)
        return ViewHolder(view)
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        val currentItem = datos[position]
        holder.bind(currentItem)
    }

    override fun getItemCount() = datos.size

    inner class ViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        // Referencias a las vistas dentro de un elemento de la lista
        private val textoTitulo: TextView = itemView.findViewById(R.id.textoTitulo)
        private val imagenIcono: ImageView = itemView.findViewById(R.id.imagenIcono)

        // Método para enlazar datos a las vistas
        fun bind(item: MiModelo) {
            textoTitulo.text = item.titulo
            // Configurar otras vistas según sea necesario
        }
    }
}
```

Vamos a inflar la vista item_pokemon

```
class PokemonAdapter {  
  
    inner class ViewHolder (view: View) : RecyclerView.ViewHolder(view) {  
  
        val binding = ItemPokemonBinding.bind(view) //esto es para poder acceder a los elementos del layout  
    }  
}
```

Un **viewholder** es una clase que se encarga de mantener los elementos de la lista en memoria

Adapter

Necesitamos añadirle un constructor y una herencia de `recyclerViewAdapter`

Añadimos al adapter lo que vamos a recibir

El constructor recibirá una lista de items pokemon y un listener que será el que hemos creado:

```
package com.tdam.recyclerview

import android.view.View
import androidx.recyclerview.widget.RecyclerView
import com.tdam.recyclerview.databinding.ItemPokemonBinding

class PokemonAdapter (var listapokemon: MutableList<Pokemon>, private val listener: OnClickListener){

    //un viewholder es una clase que se encarga de mantener los elementos de la lista en memoria
    inner class ViewHolder (view: View) : RecyclerView.ViewHolder(view) {

        val binding = ItemPokemonBinding.bind(view) //esto es para poder acceder a los elementos del layout
    }
}
```

Heredamos de RecyclerView.adapter

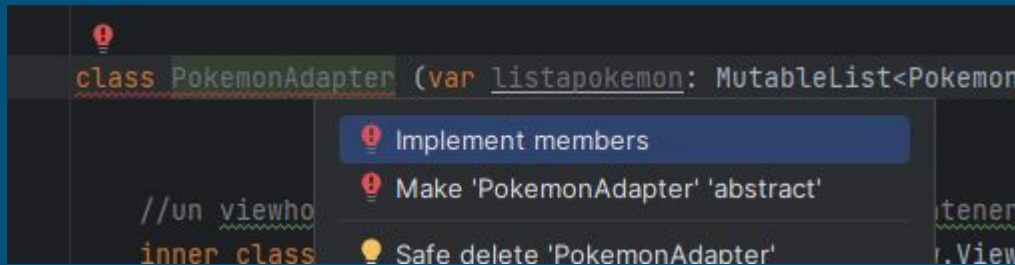
```
class PokemonAdapter (var listapokemon: MutableList<Pokemon>, private val listener: OnClickListener) : RecyclerView.Adapter<PokemonAdapter.ViewHolder>(){

    //un viewHolder es una clase que se encarga de mantener los elementos de la lista en memoria
    inner class ViewHolder (view: View) : RecyclerView.ViewHolder(view) {

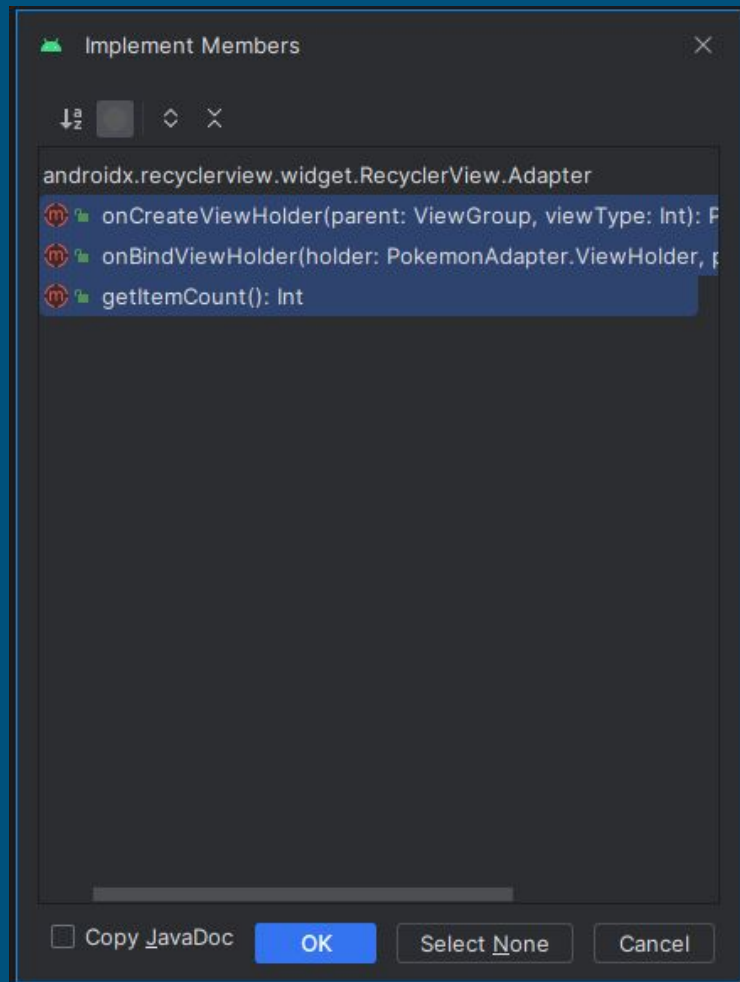
        val binding = ItemPokemonBinding.bind(view) //esto es para poder acceder a los elementos del layout
    }
}
```

Nos marcará un error porque tenemos que implementar algunos métodos para que funcione al heredar.

Resolvemos el error



```
class PokemonAdapter (var listapokemon: MutableList<Pokemon>, private val listener: OnItemClickListener) {  
    //un viewholder es una clase que se encarga de mantener los elementos de la lista en  
    inner class ViewHolder (view: View) : RecyclerView.ViewHolder(view) {  
        val binding = ItemPokemonBinding.bind(view) //esto es para poder acceder a los elementos de la lista  
    }  
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {  
        TODO("Not yet implemented")  
    }  
    override fun getItemCount(): Int {  
        TODO("Not yet implemented")  
    }  
    override fun onBindViewHolder(holder: ViewHolder, position: Int) {  
        TODO("Not yet implemented")  
    }  
}
```



getItemCount()

```
//este metodo nos indica cuantos elementos tiene la lista
override fun getItemCount(): Int {
    return listapokemon.size
}
```

0

```
//este metodo nos indica cuantos elementos tiene la lista
override fun getItemCount(): Int = listapokemon.size
```

onCreateViewHolder()

```
//este metodo se ejecuta cuando se crea el viewholder
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
    //aqui inflamamos el layout que hemos creado para cada elemento de la lista
    val view = LayoutInflater.from(parent.context).inflate(R.layout.item_pokemon, parent, attachToRoot: false)

    //aqui creamos el viewholder y le pasamos la vista que acabamos de inflar para luego devolverlo
    return ViewHolder(view)
}
```

onBindViewHolder()

```
//este metodo se ejecuta cuando se quiere mostrar un elemento de la lista
override fun onBindViewHolder(holder: ViewHolder, position: Int) {
    //le daremos valores a los elementos de cada item de la lista
    //en este caso solo tenemos un textview que será el nombre del pokemon
    val pokemon = listapokemon.get(position)

    holder.binding.textView.text = pokemon.nombre
}
```

En resumen:

```
vity_main.xml  PokemonAdapter.kt  x  MainActivity.kt  Pokemon.kt  OnClickListener.kt  </> item_pokemon.xml

package com.tdam.recyclerview

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.recyclerview.widget.RecyclerView
import com.tdam.recyclerview.databinding.ItemPokemonBinding

class PokemonAdapter (var listapokemon: MutableList<Pokemon>, private val listener: OnClickListener) : RecyclerView.Adapter<PokemonAdapter.ViewHolder>(){

    //un viewHolder es una clase que se encarga de mantener los elementos de la lista en memoria
    inner class ViewHolder (view: View) : RecyclerView.ViewHolder(view) {

        val binding = ItemPokemonBinding.bind(view) //esto es para poder acceder a los elementos del layout
    }

    //este metodo se ejecuta cuando se crea el viewHolder
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
        //aquí inflamos el layout que hemos creado para cada elemento de la lista
        val view = LayoutInflater.from(parent.context).inflate(R.layout.item_pokemon, parent, attachToRoot: false)

        //aquí creamos el viewHolder y le pasamos la vista que acabamos de inflar para luego devolverlo
        return ViewHolder(view)
    }

    //este metodo nos indica cuantos elementos tiene la lista
    override fun getItemCount(): Int = listapokemon.size

    //este metodo se ejecuta cuando se quiere mostrar un elemento de la lista
    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        //le daremos valores a los elementos de cada item de la lista
        //en este caso solo tenemos un textView que será el nombre del pokemon
        val pokemon = listapokemon.get(position)

        holder.binding.textView.text = pokemon.nombre
    }
}
```

Visualizar listado

Paso previo de buenas praxis es crear una variable global del adaptador porque se va a ver afectado desde más de un punto en toda la clase

```
5 > import ...
7
8 </> class MainActivity : AppCompatActivity(), OnClickListener {
9
10     private lateinit var binding: ActivityMainBinding
11     private lateinit var pokemonAdapter: PokemonAdapter
12
13     override fun onCreate(savedInstanceState: Bundle?) {
```



Last pasito

Añadimos elementos, creamos adaptador e inicializamos el recyclerView.

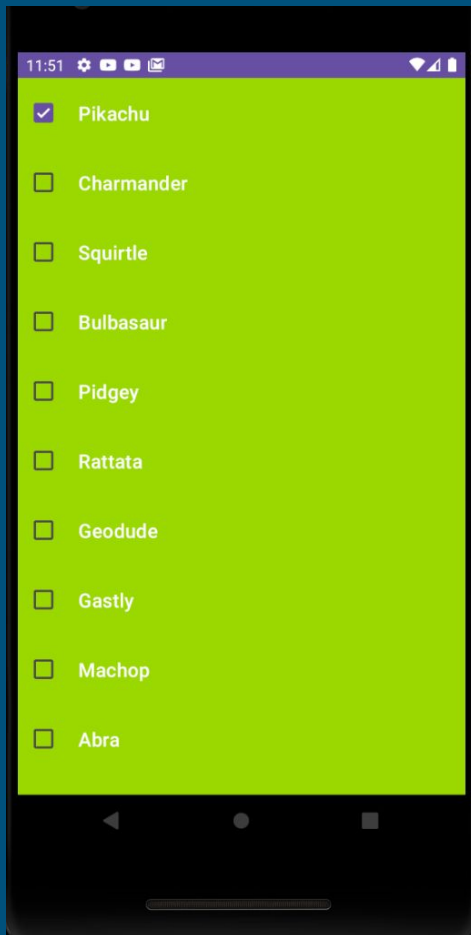
```
setContentViews(binding.root)
```

```
val data = mutableListOf(  
    Pokemon(nombre: "Pikachu", poder: 100, tipo: "Electrico", nivel: 1),  
    Pokemon(nombre: "Charmander", poder: 100, tipo: "Fuego", nivel: 1),  
    Pokemon(nombre: "Squirtle", poder: 100, tipo: "Agua", nivel: 1),  
    Pokemon(nombre: "Bulbasaur", poder: 100, tipo: "Planta", nivel: 1),  
    Pokemon(nombre: "Pidgey", poder: 100, tipo: "Volador", nivel: 1),  
    Pokemon(nombre: "Rattata", poder: 100, tipo: "Normal", nivel: 1),  
    Pokemon(nombre: "Geodude", poder: 100, tipo: "Roca", nivel: 1),  
    Pokemon(nombre: "Gastly", poder: 100, tipo: "Fantasma", nivel: 1),  
    Pokemon(nombre: "Machop", poder: 100, tipo: "Lucha", nivel: 1),  
    Pokemon(nombre: "Abra", poder: 100, tipo: "Psiquico", nivel: 1),  
    Pokemon(nombre: "Eevee", poder: 100, tipo: "Normal", nivel: 1),  
    Pokemon(nombre: "Magikarp", poder: 100, tipo: "Agua", nivel: 1),  
    Pokemon(nombre: "Snorlax", poder: 100, tipo: "Normal", nivel: 1),  
    Pokemon(nombre: "Mew", poder: 100, tipo: "Psiquico", nivel: 1),  
    Pokemon(nombre: "Mewtwo", poder: 100, tipo: "Psiquico", nivel: 1),  
    Pokemon(nombre: "Zapdos", poder: 100, tipo: "Electrico", nivel: 1),  
    Pokemon(nombre: "Articuno", poder: 100, tipo: "Hielo", nivel: 1),  
    Pokemon(nombre: "Moltres", poder: 100, tipo: "Fuego", nivel: 1),  
    Pokemon(nombre: "Dragonite", poder: 100, tipo: "Dragon", nivel: 1),  
    Pokemon(nombre: "Gyarados", poder: 100, tipo: "Agua", nivel: 1),  
    Pokemon(nombre: "Lapras", poder: 100, tipo: "Agua", nivel: 1),  
    Pokemon(nombre: "Vaporeon", poder: 100, tipo: "Agua", nivel: 1),  
    Pokemon(nombre: "Jolteon", poder: 100, tipo: "Electrico", nivel: 1),  
    Pokemon(nombre: "Flareon", poder: 100, tipo: "Fuego", nivel: 1),  
    Pokemon(nombre: "Machamp", poder: 100, tipo: "Lucha", nivel: 1),  
    Pokemon(nombre: "Alakazam", poder: 100, tipo: "Psiquico", nivel: 1),  
    Pokemon(nombre: "Gengar", poder: 100, tipo: "Fantasma", nivel: 1),  
    Pokemon(nombre: "Golem", poder: 100, tipo: "Roca", nivel: 1),  
    Pokemon(nombre: "Rhydon", poder: 100, tipo: "Tierra", nivel: 1),  
    Pokemon(nombre: "Kangaskhan", poder: 100, tipo: "Normal", nivel: 1),  
)
```

```
//al adaptador hay que pasarle la lista de datos y el listener  
pokemonAdapter = PokemonAdapter(data, listener: this) //this es el listener
```

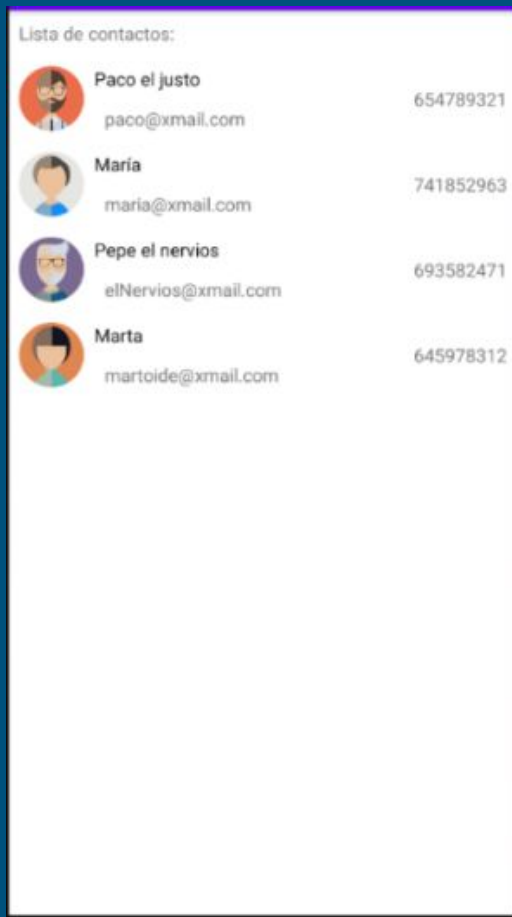
```
binding.recyclerView.apply { this: RecyclerView //aqui le decimos al recyclerView que use el adaptador que hemos creado  
    layoutManager = LinearLayoutManager(context: this@MainActivity) //esto es para que el recyclerView se muestre como una lista  
    adapter = pokemonAdapter  
}
```

Ejecutamos



Ejercicio 1

- Crear un RV con una lista de contactos
- Para cada contacto: nombre, teléfono, correo y foto
- Crear la misma interfaz que en la imagen



RV horizontal

Por defecto un RV apila sus elementos verticalmente

Para apilarlo horizontalmente debemos especificarlo a la hora de crear el LayoutManger:

```
rv.layoutManager =  
    LinearLayoutManager(context: this, LinearLayoutManager.HORIZONTAL, reverseLayout: false)
```

Ejercicio 2

Crea un RV con scroll horizontal que incluya todas las piezas de ajedrez



Añadir items

Vista para añadir

Añadimos otro elemento encima del recyclerView

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:padding="10dp">

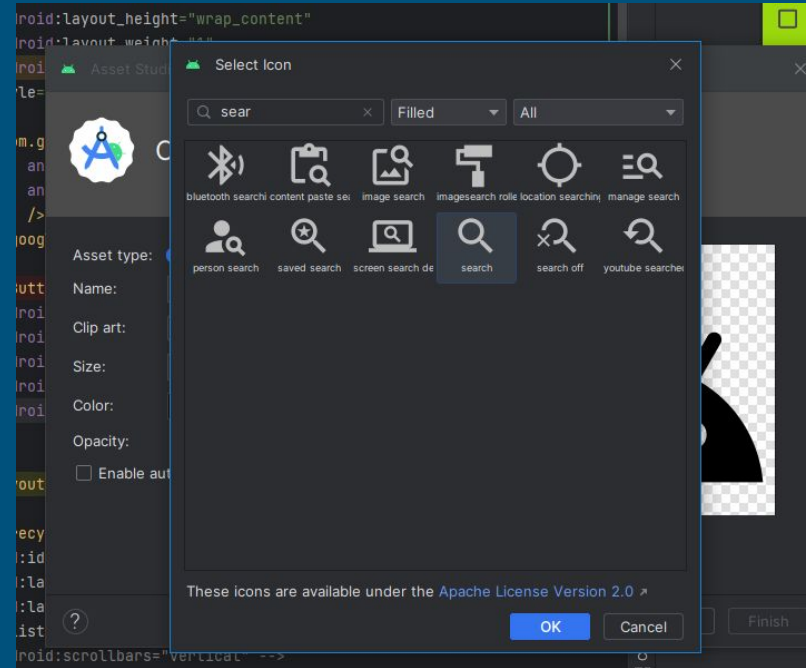
    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:hint="Escribe algo"
        style="@style/ThemeOverlay.Material3.AutoCompleteTextView.OutlinedBox"
    >
        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/etPokemon"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
        />
    </com.google.android.material.textfield.TextInputLayout>
```

Añadimos el icono de búsqueda

```
<ImageButton
```

```
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:id="@+id/btnAdd"  
    android:layout_marginStart="30dp"  
    android:src="@drawable/baseline_add"  
    android:layout_gravity="center"  
/>
```

```
</LinearLayout>
```



Escribe algo



☐ Pokemon

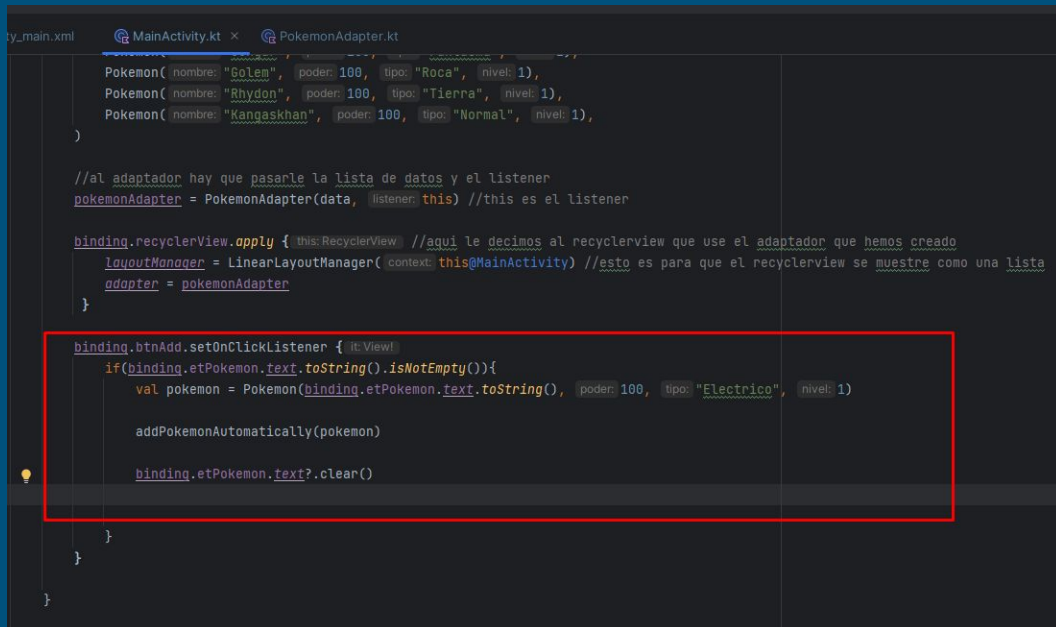
☐ Pokemon

☐ Pokemon

☐ Pokemon

Añadir pokemon en tiempo de ejecución

Primero hacemos la lógica del editText y el boton



```

y_main.xml  MainActivity.kt  PokemonAdapter.kt

Pokemon(nombre: "Golem", poder: 100, tipo: "Roca", nivel: 1),
Pokemon(nombre: "Rhydon", poder: 100, tipo: "Tierra", nivel: 1),
Pokemon(nombre: "Kangaskhan", poder: 100, tipo: "Normal", nivel: 1),
)

//al adaptador hay que pasarle la lista de datos y el listener
pokemonAdapter = PokemonAdapter(data, listener: this) //this es el listener

binding.recyclerView.apply { this:RecyclerView //aqui le decimos al recyclerview que use el adaptador que hemos creado
    layoutManager = LinearLayoutManager(context: this@MainActivity) //esto es para que el recyclerview se muestre como una lista
    adapter = pokemonAdapter
}

binding.btnAdd.setOnClickListener { it: View!
    if(binding.etPokemon.text.toString().isNotEmpty()){
        val pokemon = Pokemon(binding.etPokemon.text.toString(), poder: 100, tipo: "Electrico", nivel: 1)

        addPokemonAutomatically(pokemon)

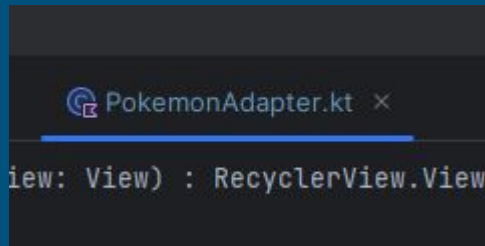
        binding.etPokemon.text?.clear()
    }
}
}

```

Declaramos la función

```
new *  
private fun addPokemonAutomatically(pokemon: Pokemon) {  
    pokemonAdapter.addPokemon(pokemon)  
}
```

Y luego creamos la función en el adaptador



notifyDataSetChanged()
se encarga de refrescar la
vista del adaptador para
que se vean los cambios.

```
fun addPokemon(pokemon: Pokemon) {  
    listapokemon.add(pokemon)  
    notifyDataSetChanged()  
}
```

Escribe algo

sadsadsasd



- ☐ Jolteon
- ☐ Flareon
- ☐ Machop
- ☐ Alakazam
- ☐ Gengar
- ☐ Golem
- ☐ Rhydon
- ☐ Kangaskhan
- ☐ dsad

Eliminar nota con click largo

Creamos un método nuevo en ViewHolder

Este método se ejecuta con respecto al ítem que esté seleccionado.

También le agregaremos un evento a la raíz de todo para que sea general al layout entero del ítem.

```
class PokemonAdapter (var listapokemon: MutableList<Pokemon>, private val listener: OnCl

//un viewholder es una clase que se encarga de mantener los elementos de la lista en
* Manuel Prieto Macias *
inner class ViewHolder (view: View) : RecyclerView.ViewHolder(view) {

    val binding = ItemPokemonBinding.bind(view) //esto es para poder acceder a los el

    new *
    fun setListener(pokemon: Pokemon) {
        binding.root.setOnLongClickListener { it: View!
            listener.onLongClick(pokemon)
            true *setOnLongClickListener
        }
    }
}
```

setemos el listener

```
//este metodo se ejecuta cuando se quiere mostrar un elemento de la lista
@ Manuel Prieto Macias *
override fun onBindViewHolder(holder: ViewHolder, position: Int) {
    //le daremos valores a los elementos de cada item de la lista
    //en este caso solo tenemos un textview que será el nombre del pokemon
    val pokemon = listapokemon.get(position)

    holder.setListener(pokemon)

    holder.binding.textView.text = pokemon.nombre
}
```

Añadimos método remove

```
new *  
fun addPokemon(pokemon: Pokemon) {  
    listapokemon.add(pokemon)  
    notifyDataSetChanged()  
}  
  
new *  
fun removePokemon(pokemon: Pokemon) {  
    listapokemon.remove(pokemon)  
    notifyDataSetChanged()  
}  
}
```



`notifyDataSetChanged()`

Este método es para que se actualice la lista

En mainActivity sobrecargamos el método

```
76
77 //sobrecargamos el metodo onLongClick de la interfaz OnClickListener
   Manuel Prieto Macías *
78 override fun onLongClick(pokemon: Pokemon) {
79     //aqui se ejecuta el codigo cuando se hace click largo en un elemento de la lista
80     removePokemonAutomatically(pokemon)
81
82 }
83
   Manuel Prieto Macías *
84 private fun removePokemonAutomatically(pokemon: Pokemon) {
85     pokemonAdapter.removePokemon(pokemon)
86
87 }
```

Añadimos un AlertDialog para prevenir liadas

Manuel Prieto Macias *

```
private fun removePokemonAutomatically(pokemon: Pokemon) {  
    val builder = AlertDialog.Builder(context: this)  
    builder.setTitle("Eliminar Pokemon")  
    builder.setMessage("¿Estas seguro de que quieres eliminar a ${pokemon.nombre}?")  
    builder.setPositiveButton(text: "Si") { _, _ ->  
        pokemonAdapter.removePokemon(pokemon)  
    }  
    builder.setNegativeButton(text: "No") { _, _ ->  
  
    }  
    builder.show()  
}
```



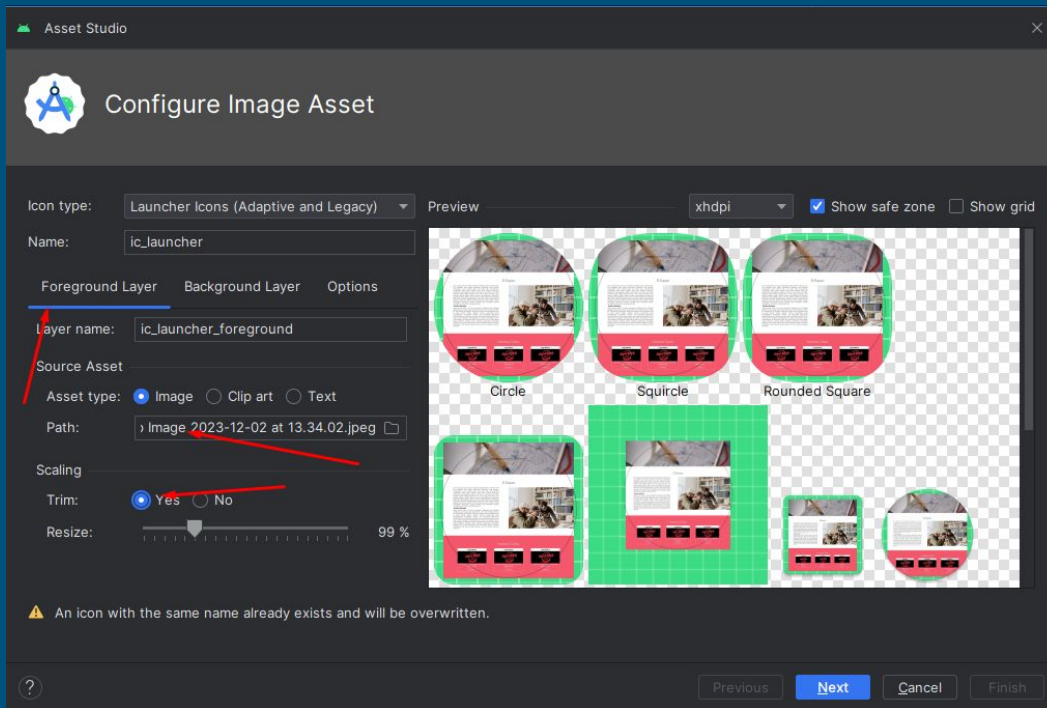
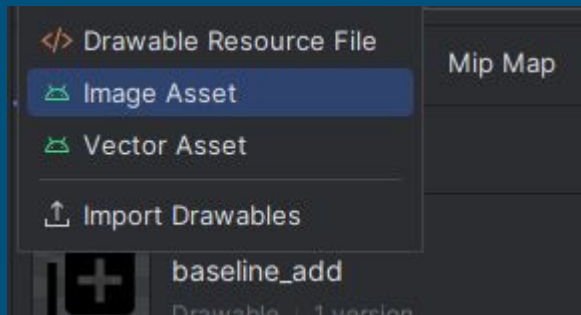
Ejercicio 3

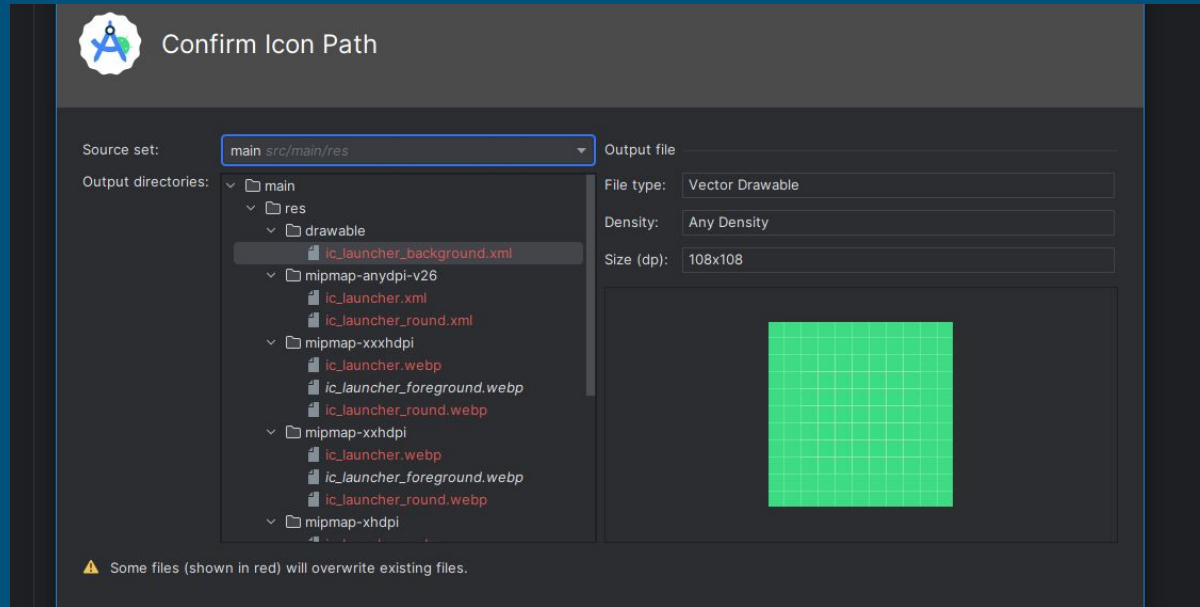
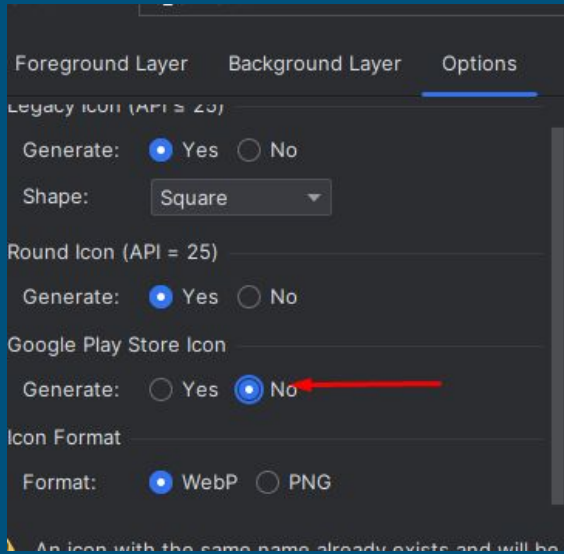
Crear el listado de pokemons o lo que querais con el estilo de cada item al gusto del consumidor.

Crear una forma de añadir items y de eliminarlos

Mejoras de UX

Creamos un icono adaptable a partir de imagen





- ▼ ic_launcher.xml
 - ▼ mipmap
 - ic_launcher.webp (hdpi)
 - ic_launcher.webp (mdpi)
 - ic_launcher.webp (xhdpi)
 - ic_launcher.webp (xxhdpi)
 - ic_launcher.webp (xxxhdpi)
 - ic_launcher.xml (anydpi-v26)
 - ▼ ic_launcher_foreground (5)
 - ic_launcher_foreground.webp (hdpi)
 - ic_launcher_foreground.webp (mdpi)
 - ic_launcher_foreground.webp (xhdpi)
 - ic_launcher_foreground.webp (xxhdpi)
 - ic_launcher_foreground.webp (xxxhdpi)
 - ▼ ic_launcher_round (6)
 - ic_launcher_round.webp (hdpi)
 - ic_launcher_round.webp (mdpi)
 - ic_launcher_round.webp (xhdpi)
 - ic_launcher_round.webp (xxhdpi)
 - ic_launcher_round.webp (xxxhdpi)
 - ic_launcher_round.xml (anydpi-v26)
 - ▼ imagen1 (5)
 - imagen1.webp (hdpi)
 - imagen1.webp (mdpi)
 - imagen1.webp (xhdpi)
 - imagen1.webp (xxhdpi)
 - imagen1.webp (xxxhdpi)
 - values



Podremos cambiar el fondo de rejilla

Iconos y colores

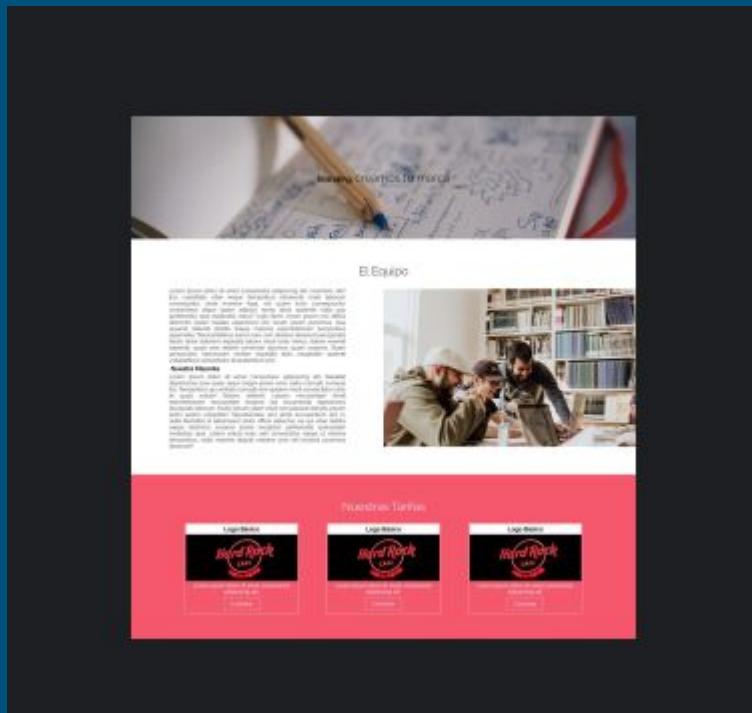
Buscamos nuestro icono

ic_launcher (6)

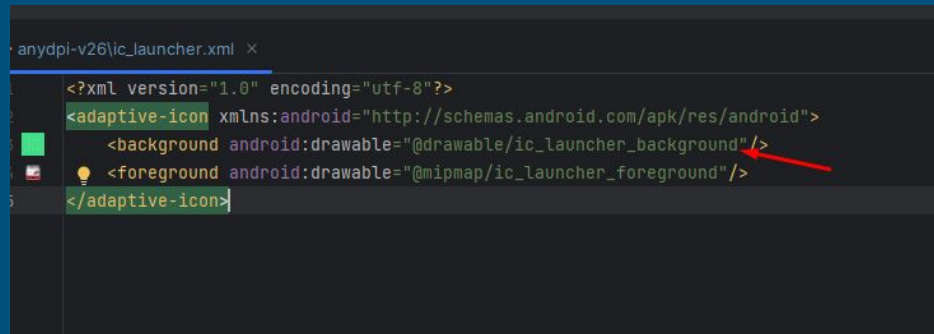
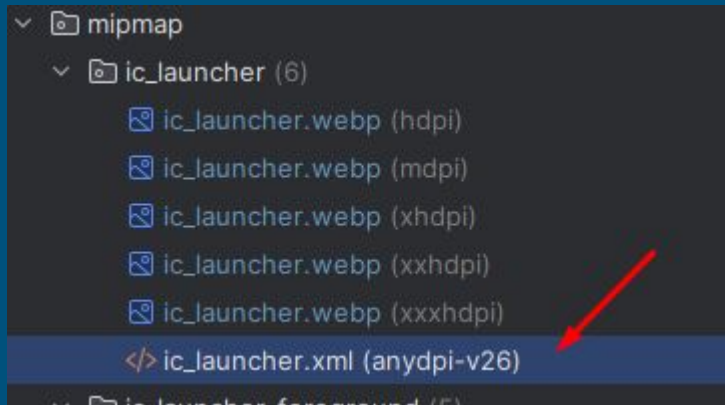
- ic_launcher.webp (hdpi)
- ic_launcher.webp (mdpi)
- ic_launcher.webp (xhdpi)
- ic_launcher.webp (xxhdpi)
- ic_launcher.webp (xxxhdpi)
- ic_launcher.xml (anydpi-v26)

ic_launcher_foreground (5)

- ic_launcher_foreground.webp (hdpi)
- ic_launcher_foreground.webp (mdpi)
- ic_launcher_foreground.webp (xhdpi)
- ic_launcher_foreground.webp (xxhdpi)
- ic_launcher_foreground.webp (xxxhdpi)



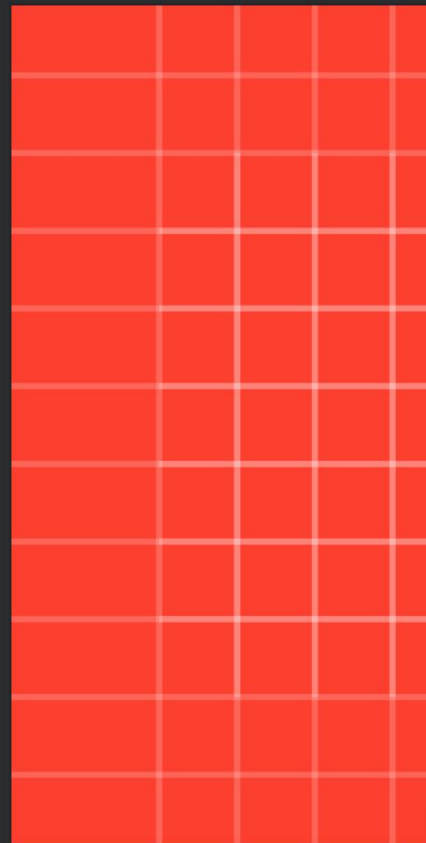
Editamos



Editamos

```
<?xml version="1.0" encoding="utf-8"?>
<vector
    android:height="108dp"
    android:width="108dp"
    android:viewportHeight="108"
    android:viewportWidth="108"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <path android:fillColor="#3DDC84"
        android:pathData="M0,0h108v108h-108z"/>
    <path android:fillColor="#00000000" android:pathData="M9,0L9,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M19,0L19,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M29,0L29,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M39,0L39,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M49,0L49,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M59,0L59,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M69,0L69,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M79,0L79,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M89,0L89,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M99,0L99,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M0,9L108,9"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M0,19L108,19"
```

```
<?xml version="1.0" encoding="utf-8"?>
<vector
    android:height="108dp"
    android:width="108dp"
    android:viewportHeight="108"
    android:viewportWidth="108"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <path android:fillColor="#fd3f2f"
        android:pathData="M0,0h108v108h-108z"/>
    <path android:fillColor="#00000000" android:pathData="M19,0L19,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M29,0L29,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M39,0L39,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M49,0L49,108"
```

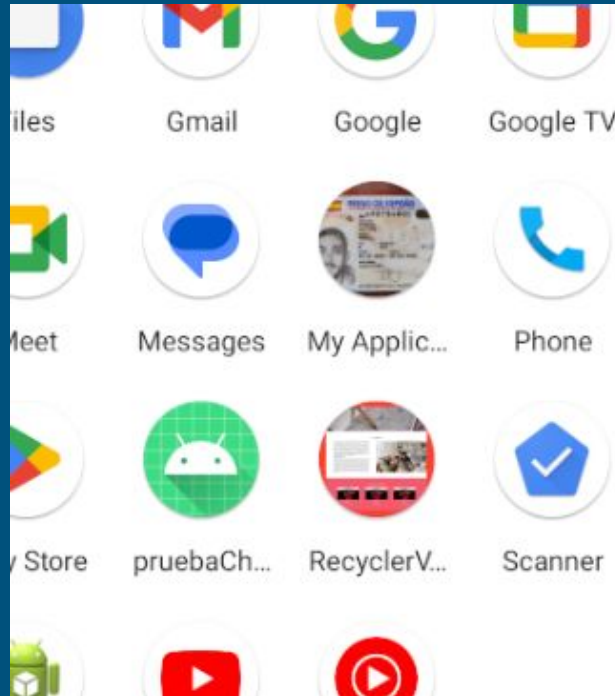


Tambien podemos eliminarlas todas

```
<?xml version="1.0" encoding="utf-8"?>
<vector
  android:height="108dp"
  android:width="108dp"
  android:viewportHeight="108"
  android:viewportWidth="108"
  xmlns:android="http://schemas.android.com/apk/res/android">
  <path android:fillColor="#fd3f2f"
        android:pathData="M0,0h108v108h-108z"/>
</vector>
```



Resultado



Material colors

<https://materialui.co/colors/>



Colors

Material

Flat UI

Social

Metro

HTML

Color Picker

Tailwind 

Fluent

More

Icons

Unicode

Emoji Keyboard 

Blog

TDFW?

About

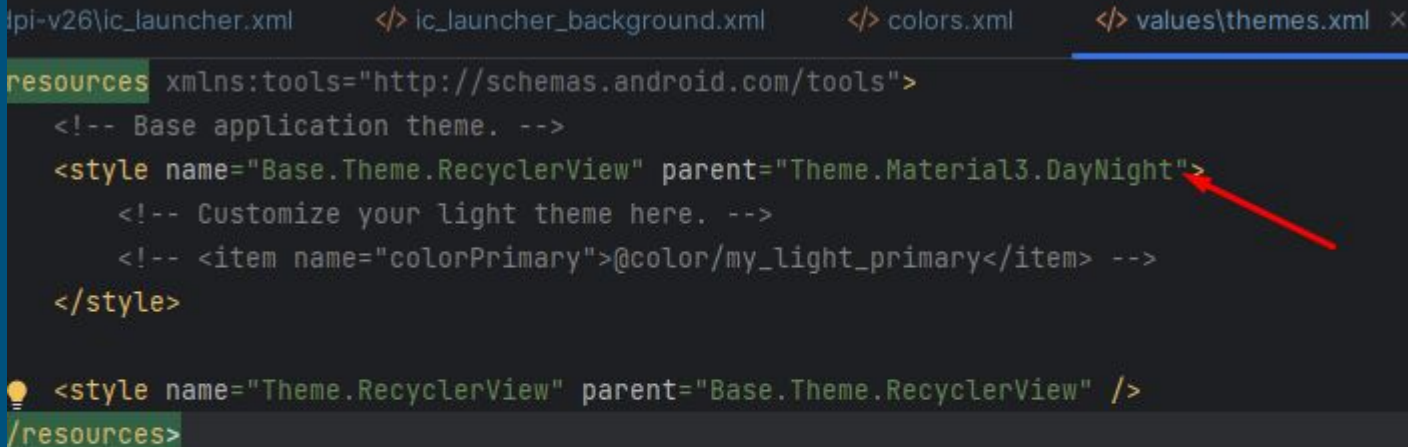
Advertise

[illegible]

Agregamos color

```
</> anydpi-v26\lc_launcher.xml    </> ic_launcher_background.xml    </> colors.xml x
1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3      <color name="black">#FF000000</color>
4      <color name="white">#FFFFFFFF</color>
5    <color name="verde">#9CCC65</color>
6  </resources>
```

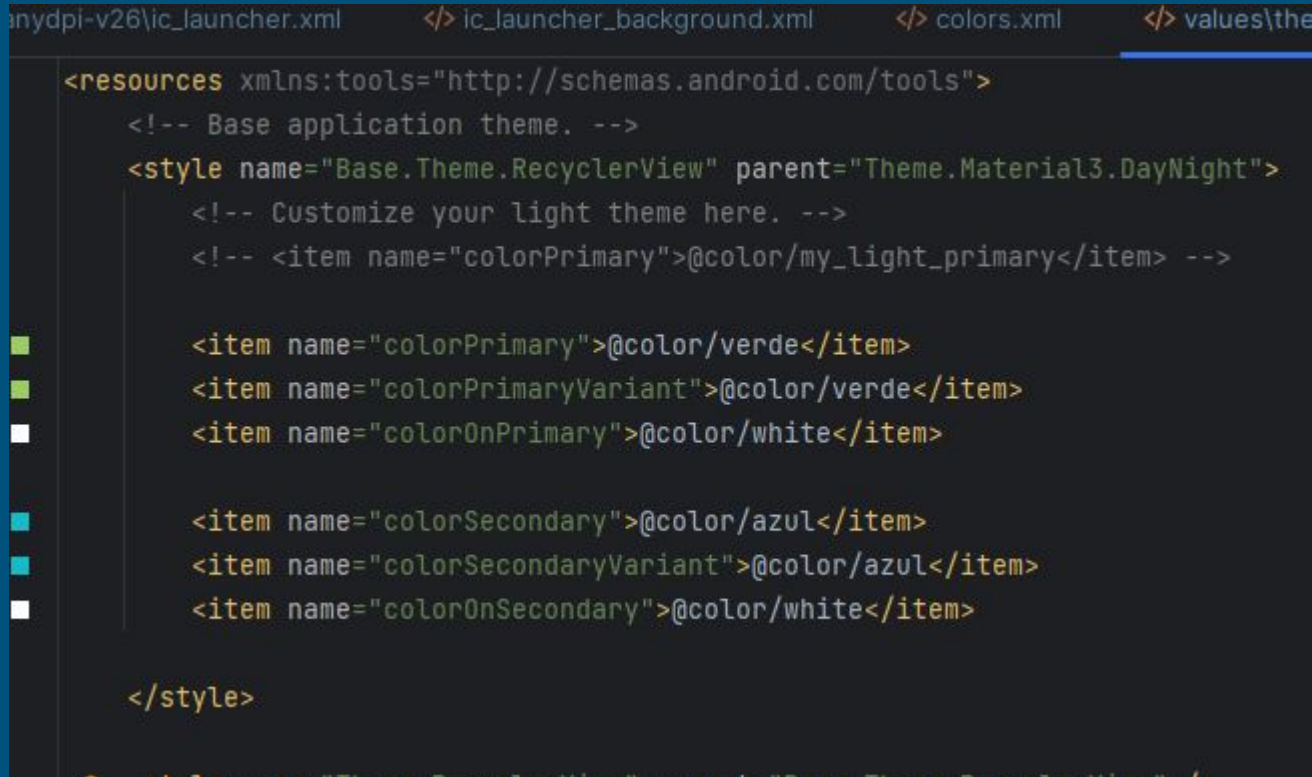
Activamos action bar



```
resources xmlns:tools="http://schemas.android.com/tools">
  <!-- Base application theme. -->
  <style name="Base.Theme.RecyclerView" parent="Theme.Material3.DayNight">
    <!-- Customize your light theme here. -->
    <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
  </style>

  <style name="Theme.RecyclerView" parent="Base.Theme.RecyclerView" />
</resources>
```

Agregamos el color base del tema



```
<?xml version="1.0" encoding="utf-8"?>
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.RecyclerView" parent="Theme.Material3.DayNight">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->

        <item name="colorPrimary">@color/verde</item>
        <item name="colorPrimaryVariant">@color/verde</item>
        <item name="colorOnPrimary">@color/white</item>

        <item name="colorSecondary">@color/azul</item>
        <item name="colorSecondaryVariant">@color/azul</item>
        <item name="colorOnSecondary">@color/white</item>

    </style>
</resources>
```

2:55



RecyclerView

Escribe algo



☐ Pikachu

☐ Charmander

☐ Squirtle

☐ Bulbasaur

☐ Pidgey

☐ Rattata

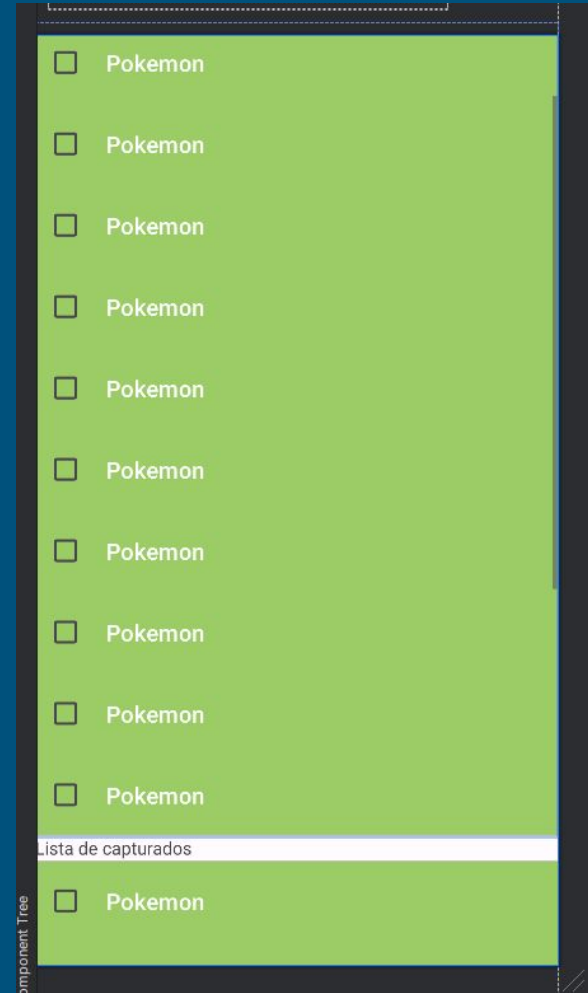
☐ Geodude

☐ Gastly

Cambiar tamaño texto desde kotlin

```
holder.binding.textView.setTextSize(TypedValue.COMPLEX_UNIT_SP, size: 12f)
```

Diseño con 2 recyclerView



Habr  que inicializar el nuevo adaptador

Que es el mismo pero con una nueva instancia

Si no lo hacemos el segundo recyclerView no funcionar 

```
Manuel Prieto Macias *
class MainActivity : AppCompatActivity(), OnClickListener {

    private lateinit var binding: ActivityMainBinding
    private lateinit var pokemonAdapter: PokemonAdapter
    private lateinit var capturadosAdapter: PokemonAdapter

    Manuel Prieto Macias
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
```

Lo inicializamos y lo asignaremos

```
Pokemon(nombre: "Rhydon", poder: 100, tipo: "Tierra", nivel: 1),
Pokemon(nombre: "Kangaskhan", poder: 100, tipo: "Normal", nivel: 1),
)

//al adaptador hay que pasarle la lista de datos y el listener
pokemonAdapter = PokemonAdapter(data, listener: this) //this es el listener
capturadosAdapter = PokemonAdapter(data, listener: this)

binding.recyclerView.apply { this: RecyclerView //aqui le decimos al recyclerview que use el adaptador que
    layoutManager = LinearLayoutManager(context: this@MainActivity) //esto es para que el recyclerview :
    adapter = pokemonAdapter
}

//añadimo es nuevo adapter
binding.recyclerView2.apply { this: RecyclerView
    layoutManager = LinearLayoutManager(context: this@MainActivity)
    adapter = capturadosAdapter
}

binding.btnAdd.setOnClickListener { it: View!
    if(binding.etPokemon.text.toString().isEmpty()){
        val pokemon = Pokemon(binding.etPokemon.text.toString(), poder: 100, tipo: "Electrico", nivel: 1)

        addPokemonAutomatically(pokemon)

        binding.etPokemon.text?.clear()
    }
}
```

RecyclerView

☐ Golem

☐ Rhydon

☐ Kangaskhan

Lista de capturados

☐ Pikachu

☐ Charmander

☐ Squirtle

☐ Bulbasaur

☐ Pidgey

☐ Rattata

Separar datos

1. ponemos una lista vacia

```
//al adaptador hay que pasarle la lista de datos y el listener  
pokemonAdapter = PokemonAdapter(mutableListOf(), listener: this) //this es el listener  
capturadosAdapter = PokemonAdapter(mutableListOf(), listener: this)
```

Creamos la función getData para poder recoger datos más dinámicamente

```
new *
private fun getData(){
    val data = mutableListOf(
        Pokemon( nombre: "Pikachu", poder: 100, tipo: "Electrico", nivel: 1),
        Pokemon( nombre: "Charmander", poder: 100, tipo: "Fuego", nivel: 1),
        Pokemon( nombre: "Squirtle", poder: 100, tipo: "Agua", nivel: 1),
        Pokemon( nombre: "Bulbasaur", poder: 100, tipo: "Planta", nivel: 1),
        Pokemon( nombre: "Pidgey", poder: 100, tipo: "Volador", nivel: 1),
        Pokemon( nombre: "Rattata", poder: 100, tipo: "Normal", nivel: 1),

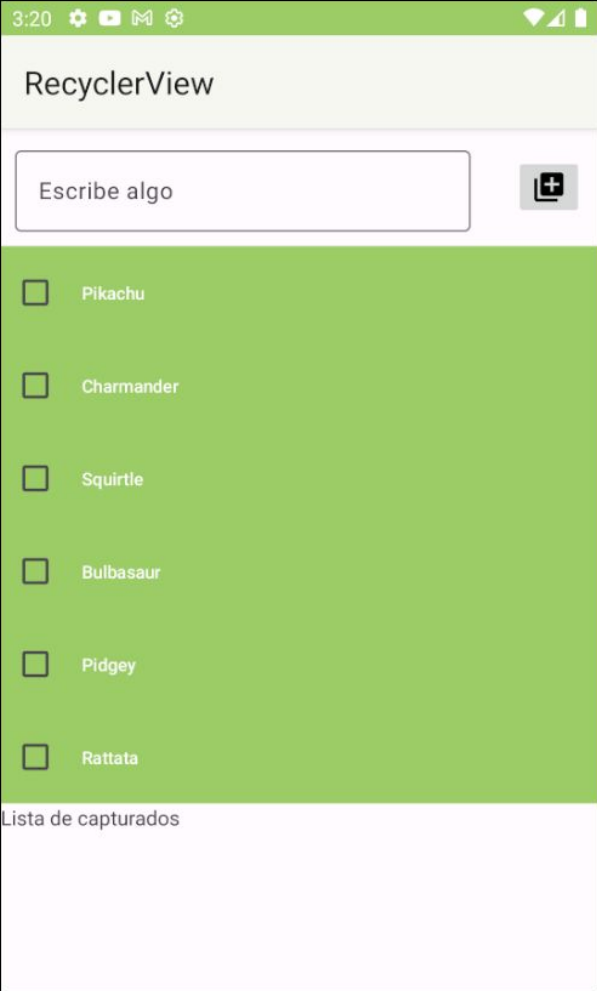
    )

    data.forEach {pokemon ->
        addPokemonAutomatically(pokemon)
    }
}
```

Sobrecargamos el estado onStart()

Para setear datos correctamente

```
new *  
override fun onStart() {  
    super.onStart()  
    getData()  
}
```



Añadimos el checkbox a capturado

```
<> anydpi-v26\ic_launcher.xml  MainActivity.kt  activity_main.xml  PokemonAdapter.kt  Pokemon.kt x  ic_launcher_background.xml  colors.xml  values\themes.xml
1 package com.tdam.recyclerview
2
3 data class Pokemon (val nombre: String = "Sin nombre", val poder: Int = 0, val tipo: String = "Normal", val nivel: Int = 0, var capturado: Boolean = false)
```

```
//este metodo se ejecuta cuando se quiere mostrar un elemento de la lista
* Manuel Prieto Macias *
override fun onBindViewHolder(holder: ViewHolder, position: Int) {
    //le daremos valores a los elementos de cada item de la lista
    //en este caso solo tenemos un textview que será el nombre del pokemon
    val pokemon = listapokemon.get(position)

    holder.setListener(pokemon)

    holder.binding.textView.text = pokemon.nombre
    holder.binding.checkBox.isChecked = pokemon.capturado
}
```

Manuel Prieto Macías *

```
private fun addPokemonAutomatically(pokemon: Pokemon) {  
  
    //si por ejemplo el nombre del pokemon es pikachu o tiene la 'a', lo añadimos al segundo adaptador  
    if (pokemon.capturado) {  
        capturadosAdapter.addPokemon(pokemon)  
    }else{  
        pokemonAdapter.addPokemon(pokemon)  
    }  
  
}
```

new *

```
private fun getData(){
```

```
    val data = mutableListOf(  
        Pokemon( nombre: "Pikachu", poder: 100, tipo: "Electrico", nivel: 1, capturado: true),  
        Pokemon( nombre: "Charmander", poder: 100, tipo: "Fuego", nivel: 1),  
        Pokemon( nombre: "Squirtle", poder: 100, tipo: "Agua", nivel: 1, capturado: true),  
        Pokemon( nombre: "Bulbasaur", poder: 100, tipo: "Planta", nivel: 1),  
        Pokemon( nombre: "Pidgey", poder: 100, tipo: "Volador", nivel: 1),  
        Pokemon( nombre: "Rattata", poder: 100, tipo: "Normal", nivel: 1),  
    )  
}
```

3:25

RecyclerView

Escribe algo



☐ Charmander

☐ Bulbasaur

☐ Pidgey

☐ Rattata

Lista de capturados

✓ Pikachu

✓ Squirtle

Añadimos un nuevo listener

```
package com.tdam.recyclerview
```

```
Manuel Prieto Macias *
```

```
interface OnClickListener {
```

```
Manuel Prieto Macias
```

```
fun onLongClick(pokemon: Pokemon) //funcion que se ejecuta cuando s  
new *
```

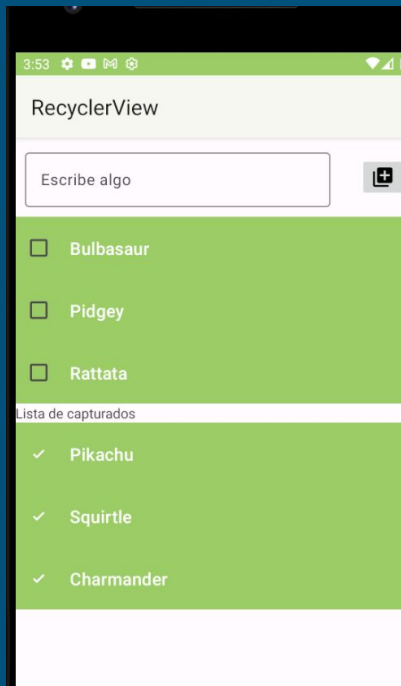
```
fun onPokemonCaptured(pokemon: Pokemon)
```

Luego, en PokemonAdapter, configuramos un listener para el checkbox dentro del método onBindViewHolder:

```
holder.binding.checkBox.setOnCheckedChangeListener { _, isChecked ->
    if (isChecked) {
        pokemon.capturado = true
        listener.onPokemonCaptured(pokemon)
    }
}
```

En MainActivity, implementamos estos métodos para mover los Pokemon entre las listas:

```
new *  
override fun onPokemonCaptured(pokemon: Pokemon) {  
    pokemonAdapter.removePokemon(pokemon)  
    capturadosAdapter.addPokemon(pokemon)  
}
```



Ejercicio 4

Hacer lo mismo pero para que cuando quite el checkbox de la lista de capturados, se pase al primer recyclerView, será como liberar a los pokemon <3